Air

Langage	**Python 3.12.3**	ML	**XGBoost / LightGBM / SHAP**
API Backend	**FastAPI + Uvicorn**	Données	**pandas (CSV en mémoire)**
Dashboard	**Streamlit**	Base de données	**Supabase PostgreSQL**
Déploiement	**Render + Docker**	Auth	**JWT (rôle unique)**

Table des matières

1 Équipe DPA Green Tech **2**

2 Pile Technologique **3**

3 Arborescence Réelle du Projet **3**
3.1 Structure détaillée de l'API — `api/` 5
3.2 Les notebooks de data science 7

4 Réconciliation des Approches **8**
4.1 Décisions clés issues de la réconciliation 8
4.2 Tableau de correspondance : approches vs réalité 9

5 Dépendances — requirements.txt Final **9**

6 Variables d'Environnement — .env **11**

7 Architecture des Données **12**
7.1 Table users — seule table PostgreSQL 13

8 Tous les Endpoints API **13**

9 Plan de Développement Backend **15**
9.1 Étape 1 — Configurer Supabase **Priorité absolue** 15
9.2 Étape 2 — api/core/config.py + api/core/database.py 15
9.3 Étape 3 — Modèle User + Migration Alembic 15
9.4 Étape 4 — api/services/data_service.py 16
9.5 Étape 5 — Chargement des modèles XGBoost/LightGBM 17
9.6 Étape 6 — IRS Service 17
9.7 Étape 7 — Routers data publics 18
9.8 Étape 8 — Auth JWT 18
9.9 Étape 9 — Chatbox Claude IA 19
9.10 Étape 10 — api/main.py final avec lifespan 19
9.11 Étape 11 — Dockerisation 20
9.12 Étape 12 — Déploiement Render.com 22

10 Lancer le Projet en Local **22**

11 Calendrier de Développement **24**

12 Impact sur les Critères du Hackathon **25**

1 Équipe DPA Green Tech

Membre	Institution	Rôle
BODEHOU Sabine	ISSEA Yaoundé	**Data Science**
FANKAM Marc Aurel	ISSEA Yaoundé	**Modélisation ML**
PEURBAR RIMBAR Firmin	ISSEA Yaoundé	**SHAP & Rapport**
FOFACK ALEMDJOU Henri Joël	ENSP Yaoundé	**Frontend & API**

2 Pile Technologique

Catégorie	Outils	Usage
Langage	Python 3.12.3	Tout le projet
Data Science	Pandas, NumPy, Scikit-learn, SciPy	Traitement données, features
Machine Learning	XGBoost, LightGBM	Prédiction PM2.5 / qualité air
Interprétabilité	SHAP	Explication des prédictions
Visualisation	Matplotlib, Seaborn, Plotly, Folium	Graphiques, carte interactive
Backend / API	FastAPI, Uvicorn	API RESTful + Swagger auto
Dashboard	Streamlit	Interface utilisateur interactive
Authentification	JWT (python-jose, passlib)	Comptes utilisateurs
Base de données	PostgreSQL / Supabase	Table users + avatars
Stockage fichiers	Supabase Storage	Photos de profil utilisateurs
Déploiement	Docker + Render.com	Conteneurisation + hébergement
PWA	`manifest.json`	Progressive Web App

3 Arborescence Réelle du Projet

Source de vérité

Cette arborescence reflète fidèlement le **README.md** du dépôt réel du projet AirSentinel, tel que développé par l'équipe DPA Green Tech. Toute référence à des chemins dans ce document utilise cette structure.

```
AirSentinel/
|
|-- api/                              # Backend FastAPI
```

```
|   `-- main.py                    # Point d'entree de l'API
    (uvicorn api.main:app)
|
|-- dashboard/                     # Frontend Streamlit
|   |-- app.py                     # Script principal (
   streamlit run dashboard/app.py)
|   |-- blocs/                     # Composants modulaires
   de l'UI (6 blocs)
|   |-- utils/                     # Fonctions utilitaires
   partagees
|   |-- assets.py                  # Gestion des assets (
   images, logos)
|   |-- chatbox.py                 # Module chat IA integre
|   |-- landing.py                 # Page d'accueil du
   dashboard
|   |-- themes.py                  # Configuration themes
   visuels (dark/light)
|   `-- translations.py            # Textes et traductions (
   FR/EN)
|
|-- data/                          # Donnees du projet
|   |-- raw/                       # Donnees brutes
   originales
|   `-- processed/                 # Donnees nettoyees et
   preparees
|
|-- deployment/                    # Configurations de
   deploiement
|   `-- render.yaml                # Config service Render.
   com
|
|-- graphiques/                    # Resultats visuels
   generes (SHAP, correlations)
|
|-- models/                        # Modeles ML entrained (
   fichiers joblib)
|   |-- meilleur_modele.joblib     # Modele XGBoost/LightGBM
    principal
|   |-- scaler_acp_irs.joblib      # Scaler pour le calcul
   IRS
|   |-- pca_irs.joblib             # ACP pour l'IRS
|   |-- cols_irs.joblib            # Colonnes utilisees par
```

```
   l'IRS
|   `-- seuils_irs.joblib          # Seuils percentiles (p50
   , p75, p90)
|
|-- notebooks/                     # Notebooks Jupyter de
   data science
|   |-- 01_chargement_nettoyage.ipynb
|   |-- 02_tests_statistiques.ipynb
|   |-- 03_feature_engineering.ipynb
|   |-- 04_modelisation.ipynb
|   |-- 05_irs_episodes.ipynb
|   `-- 06_shap_interpretation.ipynb
|
|-- pwa/                           # Configuration PWA
|   `-- manifest.json              # Manifeste Progressive
   Web App
|
|-- rapport/                       # Documents et rapports
   generes
|
|-- README.md                      # Documentation
   principale
`-- requirements.txt               # Dependances Python
```

3.1 Structure détaillée de l'API — `api/`

Le fichier `api/main.py` est le point d'entrée unique de l'API. Pour garder le code maintenable, il faut l'organiser en sous-modules à créer à l'intérieur du dossier `api/` :

```
api/
|-- __init__.py
|-- main.py                     # FastAPI app + lifespan +
   routers
|-- core/
|   |-- config.py               # Settings Pydantic (lit le
   .env)
|   `-- database.py             # SQLAlchemy async (table
   users)
|-- models/
|   `-- user.py                 # Modele SQLAlchemy User
|-- schemas/
```

```
|   |-- user.py                 # UserCreate, UserResponse,
    Token
|   |-- pollution.py            # Schemas KPIs, Carte,
    Analyses
|   |-- prediction.py           # Schemas predictions,
    simulateur
|   |-- filters.py              # Query params communs
|   `-- chat.py                 # ChatRequest, ChatResponse
|-- auth/
|   |-- router.py               # /auth/register, /login, /
    refresh, /me
|   |-- service.py              # hash_password,
    create_token
|   `-- dependencies.py         # get_current_user()
|-- routers/
|   |-- kpis.py                 # GET /kpis
|   |-- carte.py                # GET /carte + /carte/
    analyses
|   |-- predictions.py          # GET /predictions/* + POST
    /simulate-irs
|   |-- alertes.py              # GET /alertes
|   |-- decision.py             # GET /decision/real
|   |-- contexte.py             # GET /contexte
|   |-- chat.py                 # POST /chat (proxy Claude
    API)
|   `-- users.py                # POST /users/me/avatar
`-- services/
    |-- data_service.py         # Chargement CSV + filtrage
    pandas
    |-- irs_service.py          # Calcul IRS, percentiles,
    niveaux
    |-- prediction_service.py   # Inference joblib + logique
     72h
    `-- alert_service.py        # Logique alertes par niveau
     IRS
```

```
# Commandes pour creer la structure dans api/
cd AirSentinel
mkdir -p api/core api/models api/schemas api/auth api/
   routers api/services
touch api/__init__.py api/core/__init__.py api/models/
```

```
    __init__.py
touch api/schemas/__init__.py api/auth/__init__.py
touch api/routers/__init__.py api/services/__init__.py
touch api/core/config.py api/core/database.py
touch api/models/user.py
touch api/schemas/user.py api/schemas/pollution.py
touch api/schemas/prediction.py api/schemas/filters.py api/
    schemas/chat.py
touch api/auth/router.py api/auth/service.py api/auth/
    dependencies.py
touch api/routers/kpis.py api/routers/carte.py api/routers/
    predictions.py
touch api/routers/alertes.py api/routers/decision.py
touch api/routers/contexte.py api/routers/chat.py api/
    routers/users.py
touch api/services/data_service.py api/services/irs_service
    .py
touch api/services/prediction_service.py api/services/
    alert_service.py
```

3.2 Les notebooks de data science

Les 6 notebooks documentent le pipeline complet de modélisation :

Notebook	Contenu
`01_chargement_nettoyage`	Import dataset, valeurs manquantes, types, export `processed/`
`02_tests_statistiques`	Tests de normalité, corrélations, distributions par région
`03_feature_engineering`	Création variables IRS, encodage, saisonnalité
`04_modelisation`	Entraînement XGBoost/LightGBM, validation croisée, métriques
`05_irs_episodes`	Calcul IRS (ACP pondérée), détection épisodes climatiques
`06_shap_interpretation`	Valeurs SHAP, importance variables, export `graphiques/`

4 Réconciliation des Approches

Principe fondamental

L'approche architecture (Guide de Démarrage) apporte les bonnes pratiques — auth JWT, Supabase, Alembic, Docker. La spec AirSentinel apporte les endpoints métier précis. Le README réel fixe l'arborescence définitive. **Les trois se complètent sans se contredire.**

4.1 Décisions clés issues de la réconciliation

1. **Dossier API = `api/` (pas `app/`).** Conformément au README. Le point d'entrée est `api/main.py`, lancé avec `uvicorn api.main:app`. Tous les imports utilisent le préfixe `api.` : `from api.core.config import get_settings`.
2. **Modèles ML = fichiers `.joblib` dans `models/`.** Le dossier `models/` est à la **racine** du projet. Les modèles sont XGBoost/LightGBM entraînés par Sabine et Marc Aurel, sérialisés avec `joblib`. Chemins : `models/meilleur_modele.joblib`, `models/scaler_acp_irs.joblib`, etc.
3. **Données core = CSV dans `data/processed/`.** Le dataset nettoyé est dans `data/processed/`. La variable `DATASET_PATH` dans le `.env` pointe vers ce fichier.
4. **Dashboard = Streamlit (pas Next.js).** Le frontend est `dashboard/app.py`, lancé avec `streamlit run dashboard/app.py`. La structure en `blocs/` correspond aux 6 blocs de la spec AirSentinel.
5. **Déploiement = Render + Docker.** Le fichier `deployment/render.yaml` configure le service Render.com. Docker reste utilisé pour le déploiement local et la reproductibilité.
6. **Auth JWT + Supabase = conservés.** Table `users` dans Supabase PostgreSQL, bucket `profile-pictures` pour les avatars.
7. **Endpoints data = publics.** Les routes `/kpis`, `/carte`, `/predictions/*` etc. sont sans token — pour les **+10 pts bonus**.

4.2 Tableau de correspondance : approches vs réalité

Aspect	Approche Architecture	Spec AirSentinel	README réel
Dossier API	`app/`	`airsentinel-api/`	`api/`
Modèles ML	`app/ml/models/*.pk`	`app/ml/models/*.pk`	`models/*.joblib`
Dataset	`data/`	`data/`	`data/processed/`
Frontend	Next.js	Next.js	**Streamlit**
Déploiement	Docker seul	Docker seul	**Render + Docker**
Algo ML	Générique .joblib	Générique .pkl	**XGBoost / LightGBM**

5 Dépendances — requirements.txt Final

```
# -- Framework API
   ------------------------------------------------------------

fastapi==0.115.0
uvicorn[standard]==0.30.6

# -- Validation
   ------------------------------------------------------------------

pydantic[email]==2.8.2
pydantic-settings==2.4.0

# -- Base de donnees (table users uniquement)
   ------------------------------
sqlalchemy[asyncio]==2.0.35
asyncpg==0.30.0
alembic==1.13.3

# -- Auth JWT
   ------------------------------------------------------------------

python-jose[cryptography]==3.3.0
passlib[bcrypt]==1.7.4
```

```
python-multipart==0.0.12     # upload avatar

# -- Supabase Storage
   -------------------------------------------------------
supabase==2.9.0

# -- Data Science & ML
   -------------------------------------------------------
pandas==2.2.3
numpy==1.26.4
scipy==1.13.1
scikit-learn==1.5.2
xgboost==2.1.1
lightgbm==4.5.0
shap==0.46.0
joblib==1.4.2

# -- Visualisation
   ---------------------------------------------------------------

matplotlib==3.9.2
seaborn==0.13.2
plotly==5.24.1
folium==0.17.0

# -- Dashboard Frontend
   -------------------------------------------------------
streamlit==1.38.0

# -- Claude API (chatbox IA)
   ------------------------------------------------
anthropic==0.34.0

# -- Predictions temporelles 72h (optionnel)
   --------------------------------
# prophet==1.1.5     # Si Prophet disponible, sinon
   regression lineaire

# -- Utilitaires
   ---------------------------------------------------------------

python-dotenv==1.0.1
```

```
httpx==0.27.2
```

6 Variables d'Environnement — .env

```
# -- Application
   ------------------------------------------------------------

DEBUG=True
API_PREFIX=/api/v1
ALLOWED_ORIGINS=["http://localhost:8501","https://
   airsentinel.onrender.com"]

# -- Dataset (chemin dans data/processed/)
   --------------------------------
DATASET_PATH=data/processed/airsentinel_dataset.csv

# -- Modeles ML (racine models/)
   ------------------------------------------
ML_MODELS_PATH=models/

# -- Supabase PostgreSQL (table users)
   -------------------------------------
DATABASE_URL=postgresql+asyncpg://postgres.XXXX:PWD@pooler.
   supabase.com:6543/postgres
DATABASE_URL_SYNC=postgresql://postgres.XXXX:PWD@db.XXXX.
   supabase.co:5432/postgres

# -- Supabase Storage (bucket avatars)
   ------------------------------------
SUPABASE_URL=https://XXXX.supabase.co
SUPABASE_KEY=eyJhbGci...          # anon public key
SUPABASE_SERVICE_KEY=eyJhbGci... # service_role key (upload
   )

# -- JWT
   ---------------------------------------------------------------

SECRET_KEY=generer_avec_openssl_rand_hex_32
ALGORITHM=HS256
ACCESS_TOKEN_EXPIRE_MINUTES=60
```

```
REFRESH_TOKEN_EXPIRE_DAYS=7

# -- Claude API (chatbox)
   ------------------------------------------------
ANTHROPIC_API_KEY=sk-ant-...
CLAUDE_MODEL=claude-sonnet-4-20250514
```

7 Architecture des Données

Source	Contenu	Chemin réel	Persistance
CSV pandas	87 240 obs. météo, IRS, PM2.5, 42 villes	`data/processed/`	RAM (boot)
PostgreSQL Supabase	Table `users` (auth + avatar_url)	Supabase cloud	Permanente
Supabase Storage	Photos de profil (.jpg, .png, .webp)	Bucket `profile-pictures`	Permanente
Fichiers joblib	XGBoost/I + PCA + scaler IRS	`models/*.joblib`	Fichiers locaux

7.1 Table users — seule table PostgreSQL

Colonne	Type SQL	Description
id	UUID	Clé primaire auto-générée
email	VARCHAR(255)	Unique, indexé
full_name	VARCHAR(100)	Nom complet
hashed_password	VARCHAR(255)	Hash bcrypt
avatar_url	TEXT	URL publique Supabase Storage
is_active	BOOLEAN	Actif par défaut
created_at	TIMESTAMPTZ	Auto : NOW()
updated_at	TIMESTAMPTZ	Auto : ON UPDATE NOW()

8 Tous les Endpoints API

Base URL dev : http://localhost:8000/api/v1
Base URL prod : https://airsentinel-api.onrender.com/api/v1

Méthode	Endpoint	Description	Auth
Santé			
GET	`/health`	Healthcheck Docker / Render	✘
Données core — publiques			
GET	`/kpis`	6 KPIs nationaux (PM2.5, IRS, villes, tendance)	✘
GET	`/carte`	Position + PM2.5 + IRS par ville (Scattergeo)	✘
GET	`/carte/analyses`	6 analyses enrichies Bloc 1	✘
GET	`/predictions/short-term`	Historique 21j + prédiction 72h (XGBoost)	✘
GET	`/predictions/monthly`	PM2.5 mensuel par année	✘
POST	`/predictions/simulate-irs`	Simulateur IRS par paramètres météo	✘
GET	`/alertes`	Distribution jours par niveau IRS / année	✘
GET	`/decision/real`	Recommandations santé par profil	✘
GET	`/contexte`	Données saisonnières, UV/Ozone, comparaison N-1	✘
POST	`/chat`	Proxy chatbox Claude IA spécialisée	✘
Auth & Profil			
POST	`/auth/register`	Créer un compte utilisateur	✘
POST	`/auth/login`	Connexion → access + refresh token	✘
POST	`/auth/refresh`	Rafraîchir le token d'accès	✘
GET	`/auth/me`	Profil de l'utilisateur connecté	✔
POST	`/users/me/avatar`	Upload photo profil → Supabase Storage	✔

9 Plan de Développement Backend

Règle d'or

Chaque étape se teste sur Swagger avant de passer à la suivante. `http://localhost:8000/api/v1/docs` est ta boussole permanente.

9.1 Étape 1 — Configurer Supabase

Priorité absolue

Bloquant

Sans Supabase configuré, la migration Alembic échoue. Les endpoints data (pandas) fonctionnent sans DB, mais l'auth JWT nécessite la table `users`.

1. supabase.com → nouveau projet → région **EU West**.
2. **Connection String** : `Settings` → `Database` → `Transaction Pooler (port 6543)`. Ajouter `+asyncpg` après `postgresql`.
3. **Direct Connection** (port 5432) pour Alembic.
4. **Clés API** : `Settings` → `API` → `anon public + service_role`.
5. **Bucket Storage** : `Storage` → `New Bucket` → `"profile-pictures"`. Mode : Public. Types : jpeg, png, webp. Max : 2 MB.
6. Remplir `.env` et générer la SECRET_KEY : `openssl rand -hex 32`.

9.2 Étape 2 — api/core/config.py + api/core/database.py

`api/core/config.py` : classe `Settings` Pydantic lisant le `.env`. Inclure `DATASET_PATH`, `ML_MODELS_PATH`, `ANTHROPIC_API_KEY`, `CLAUDE_MODEL`.

`api/core/database.py` : moteur SQLAlchemy async, session factory `get_db()`.

Préfixe des imports

Tous les imports utilisent désormais le préfixe `api.` : `from api.core.config import get_settings`, `from api.core.database import get_db`, etc.

9.3 Étape 3 — Modèle User + Migration Alembic

```
# Apres avoir ecrit api/models/user.py :
alembic revision --autogenerate -m "create users table"
alembic upgrade head
```

```
# Verifier dans Supabase Table Editor : table "users"
    visible
```

Alembic et connexion async

Utiliser `DATABASE_URL_SYNC` (port 5432, sans `+asyncpg`) dans `alembic/env.py`. Alembic ne supporte pas les drivers async pour la génération des migrations.

9.4 Étape 4 — api/services/data_service.py

```
# api/services/data_service.py
import pandas as pd
from pathlib import Path
from api.core.config import get_settings

_df: pd.DataFrame | None = None

def get_dataframe() -> pd.DataFrame:
    """Charge le CSV une seule fois (data/processed/)."""
    global _df
    if _df is None:
        _df = pd.read_csv(
            Path(get_settings().DATASET_PATH),
            parse_dates=["date"]
        )
    return _df

def apply_filters(df, villes=None, regions=None,
                  annee_min=2020, annee_max=2025):
    if villes:
        df = df[df["city"].isin(villes)]
    if regions:
        df = df[df["region"].isin(regions)]
    return df[df["date"].dt.year.between(annee_min,
   annee_max)]
```

Noms de colonnes du CSV

Dès réception du fichier `data/processed/`, lancer `df.columns.tolist()` pour

> confirmer les noms exacts (ex : `city` vs `ville`, `pm25` vs `pm2_5`). Adapter `data_service.py` avant d'écrire les routers.

9.5 Étape 5 — Chargement des modèles XGBoost/LightGBM

```
# api/services/prediction_service.py
import joblib
from pathlib import Path
from api.core.config import get_settings

_models: dict = {}

def load_all_models() -> None:
    """Charge les joblib depuis models/ au demarrage."""
    base = Path(get_settings().ML_MODELS_PATH)
    _models["modele"]  = joblib.load(base / "
   meilleur_modele.joblib")
    _models["scaler"]  = joblib.load(base / "scaler_acp_irs
   .joblib")
    _models["pca"]     = joblib.load(base / "pca_irs.joblib
   ")
    _models["cols"]    = joblib.load(base / "cols_irs.
   joblib")
    _models["seuils"]  = joblib.load(base / "seuils_irs.
   joblib")

def get_model(name: str):
    return _models[name]
```

9.6 Étape 6 — IRS Service

L'IRS (Indice de Risque Sanitaire) est calculé via l'ACP entraînée par Firmin (notebook `05_irs_episodes.ipynb`). Les seuils de niveau (FAIBLE, MODÉRÉ, ÉLEVÉ, CRITIQUE) sont tirés des percentiles p50, p75, p90 calculés sur le sous-ensemble filtré.

Variable	Poids	Niveau	Condition	Couleur
PM2.5	35%	FAIBLE	IRS < p50	vert
Dust	25%	MODÉRÉ	p50 ≤ IRS < p75	ambre
CO	20%	ÉLEVÉ	p75 ≤ IRS < p90	coral
UV	12%	CRITIQUE	IRS ≥ p90	rouge
Ozone	8%			

9.7 Étape 7 — Routers data publics

Ordre de développement (du plus simple au plus complexe) :

1. `GET /kpis` — PM2.5 moyen, IRS moyen, n_villes_depasse_oms, polluant dominant, tendance, observations.
2. `GET /carte` — une ligne par ville avec lat/lon, pm25_moyen, irs_moyen, irs_label, irs_color.
3. `GET /carte/analyses` — PM2.5 par région, tendance 12 mois, top 3 polluants, top 5 villes critiques, épisodes, vs OMS.
4. `GET /alertes` — distribution jours par niveau IRS / année.
5. `GET /predictions/monthly` — PM2.5 mensuel.
6. `POST /predictions/simulate-irs` — calcul IRS simulé.
7. `GET /predictions/short-term` — historique 21j + prédiction XGBoost 72h.
8. `GET /decision/real` — recommandations par profil.
9. `GET /contexte` — donut, comparaison N-1, UV/Ozone.

9.8 Étape 8 — Auth JWT

```python
# api/auth/service.py
from passlib.context import CryptContext
from jose import jwt
from datetime import datetime, timedelta
from api.core.config import get_settings

pwd_ctx = CryptContext(schemes=["bcrypt"], deprecated="auto
    ")

def hash_password(pwd: str) -> str:
    return pwd_ctx.hash(pwd)
```

```python
def verify_password(plain: str, hashed: str) -> bool:
    return pwd_ctx.verify(plain, hashed)

def create_token(data: dict, expires: timedelta) -> str:
    return jwt.encode(
        {**data, "exp": datetime.utcnow() + expires},
        get_settings().SECRET_KEY,
        algorithm=get_settings().ALGORITHM
    )
```

9.9 Étape 9 — Chatbox Claude IA

```python
# api/routers/chat.py
import anthropic
from api.core.config import get_settings

SYSTEM_PROMPT = """Tu es l'assistant IA d'AirSentinel
   Cameroun.
Tu es specialise dans la qualite de l'air, les risques
   sanitaires
et les donnees climatiques du Cameroun. Reponds en francais
ou en anglais selon l'utilisateur."""

async def chat_endpoint(request: ChatRequest):
    client = anthropic.Anthropic(
        api_key=get_settings().ANTHROPIC_API_KEY
    )
    resp = client.messages.create(
        model=get_settings().CLAUDE_MODEL,
        max_tokens=1000,
        system=SYSTEM_PROMPT,
        messages=request.messages,
    )
    return {"reply": resp.content[0].text,
            "model": get_settings().CLAUDE_MODEL}
```

9.10 Étape 10 — api/main.py final avec lifespan

```python
# api/main.py
```

```python
from contextlib import asynccontextmanager
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from api.core.config import get_settings
from api.core.database import engine, Base
from api.services.data_service import get_dataframe
from api.services.prediction_service import load_all_models
# imports de tous les routers ...

settings = get_settings()

@asynccontextmanager
async def lifespan(app: FastAPI):
    get_dataframe()       # charge data/processed/ en RAM
    load_all_models()     # charge models/*.joblib
    async with engine.begin() as conn:
        await conn.run_sync(Base.metadata.create_all)
    yield

app = FastAPI(
    title="AirSentinel Cameroun API",
    description="API de prediction qualite de l'air au
   Cameroun",
    version="1.0.0",
    docs_url=f"{settings.API_PREFIX}/docs",
    redoc_url=f"{settings.API_PREFIX}/redoc",
    lifespan=lifespan,
)

app.add_middleware(CORSMiddleware,
    allow_origins=settings.ALLOWED_ORIGINS,
    allow_methods=["*"], allow_headers=["*"])

# Lancement : uvicorn api.main:app --reload
```

9.11 Étape 11 — Dockerisation

```dockerfile
# Dockerfile (racine du projet AirSentinel/)
FROM python:3.12-slim AS builder
WORKDIR /app
```

```
RUN apt-get update && apt-get install -y --no-install-
   recommends \
    gcc libpq-dev && rm -rf /var/lib/apt/lists/*
COPY requirements.txt .
RUN pip install --upgrade pip && \
    pip install --prefix=/install -r requirements.txt

FROM python:3.12-slim AS production
WORKDIR /app
COPY --from=builder /install /usr/local
COPY ./api ./api
COPY ./data/processed ./data/processed
COPY ./models ./models
COPY ./pwa ./pwa
RUN useradd -m -u 1001 apiuser && chown -R apiuser:apiuser
   /app
USER apiuser
EXPOSE 8000
HEALTHCHECK --interval=30s --timeout=10s CMD \
    python -c "import httpx; httpx.get('http://localhost
   :8000/health')"
CMD ["uvicorn", "api.main:app", "--host", "0.0.0.0", "--
   port", "8000"]
```

```
# docker-compose.yml
services:
  api:
    build: .
    container_name: airsentinel_api
    env_file: .env
    ports:
      - "8000:8000"
    volumes:
      - ./models:/app/models:ro           # joblib en
   lecture seule
      - ./data/processed:/app/data/processed:ro  # dataset
  dashboard:
    image: python:3.12-slim
    container_name: airsentinel_dashboard
    env_file: .env
    ports:
```

```
      - "8501:8501"
    volumes:
      - ./dashboard:/app/dashboard
      - ./data:/app/data:ro
      - ./models:/app/models:ro
    command: streamlit run dashboard/app.py --server.port
   8501
```

9.12 Étape 12 — Déploiement Render.com

```
# deployment/render.yaml
services:
  - type: web
    name: airsentinel-api
    env: python
    buildCommand: pip install -r requirements.txt
    startCommand: uvicorn api.main:app --host 0.0.0.0 --
   port $PORT
    envVars:
      - key: DEBUG
        value: false
      - key: DATABASE_URL
        sync: false
      - key: ANTHROPIC_API_KEY
        sync: false
```

Lancer le Dashboard Streamlit

En développement local, le dashboard est lancé indépendamment de l'API : `streamlit run dashboard/app.py`
Il sera accessible sur `http://localhost:8501`. Pour la production, Render propose aussi l'hébergement de services Streamlit.

10 Lancer le Projet en Local

```
# 1. Cloner et aller dans le dossier
git clone <URL_DU_DEPOT>
cd AirSentinel
```

```
# 2. Creer l'environnement virtuel
python3 -m venv venv
source venv/bin/activate

# 3. Installer les dependances
pip install -r requirements.txt

# 4. Configurer le .env (copier le template)
cp .env.example .env
# Remplir les valeurs Supabase + JWT + Anthropic

# 5. Appliquer la migration Alembic (cree la table users)
alembic upgrade head

# 6. Lancer l'API FastAPI (terminal 1)
uvicorn api.main:app --reload
# Swagger : http://localhost:8000/api/v1/docs

# 7. Lancer le Dashboard Streamlit (terminal 2)
streamlit run dashboard/app.py
# Dashboard : http://localhost:8501
```

11 Calendrier de Développement

Jour	Partie	Objectif	Livrable
1	Matin	Config Supabase + `api/core/config.py` + `database.py`	`.env` complet
1	Après-midi	Modèle User + migration Alembic + `data_service.py`	Table `users` dans Supabase
2	Matin	`irs_service.py` + chargement `models/*.joblib`	Calcul IRS fonctionnel
2	Après-midi	`GET /kpis` + `GET /carte` + `GET /carte/analyses`	3 endpoints sur Swagger
3	Matin	Auth JWT complet (register, login, refresh, me)	`/auth/*` testés
3	Après-midi	`/predictions/*` + `/alertes` + `/decision/real`	5 endpoints testés
4	Matin	`/contexte` + `/chat` + upload avatar	API complète
4	Après-midi	`main.py` final + Docker + tests Swagger complets	**Docker OK**
5+	—	Dashboard Streamlit (consomme l'API)	**PWA déployée**

12 Impact sur les Critères du Hackathon

Critère d'évaluation	Points	Couvert par
Performance technique (modèle, code, innovation)	35 pts	XGBoost/LightGBM + SHAP + IRS
UX & Dashboard (clarté, interactivité)	20 pts	Dashboard Streamlit + tous endpoints data
Pertinence sociétale (impact, faisabilité)	25 pts	`/decision/real` + profils santé
Communication (pitch, démo)	15 pts	Swagger + vidéo démo
Documentation (reproductibilité)	5 pts	README + Swagger auto FastAPI
Bonus : API publique + PWA	**+10 pts**	Endpoints publics + `pwa/manifest.json`

Atouts de l'équipe DPA Green Tech pour le jury

- **Modèles interprétables** — SHAP values par Firmin, impact de chaque variable météo sur les prédictions (notebook 06).
- **XGBoost + LightGBM** — deux algorithmes comparés, le meilleur sélectionné et sérialisé (`meilleur_modele.joblib`).
- **API publique documentée** — Swagger sur `/api/v1/docs`, 15 endpoints avec schémas et exemples.
- **Chatbox IA Claude** — assistant spécialisé qualité de l'air au Cameroun (`POST /chat`).
- **PWA prête** — `pwa/manifest.json` déjà présent pour l'installabilité sur mobile.
- **Déploiement Render** — `deployment/render.yaml` configuré pour la mise en production rapide.

AirSentinel Cameroun • Hackathon IndabaX Cameroon 2026 • DPA Green Tech · ISSEA · ENSP